IMPLEMENTATION RUNBOOK

Jenkins + AWS Single-Account Delivery Platform

Existing Jenkins EC2 | Hyderabad region ap-south-2 | Dev / QA / Production | EKS high availability

Design baseline
Keep the existing Jenkins EC2 instance and working CI pipeline. Add a controlled CD path using short-lived STS role sessions. Keep Dev, QA and Production in one AWS account with separate deploy roles, scoped permissions and environment-specific Kubernetes namespaces or clusters.

This guide is a deployment blueprint, not a claim that your current Jenkins, VPC, EKS cluster, buckets, IAM roles or application manifests already exist. Inventory first, preserve the working CI system, then apply changes in small reversible stages.

Prepared: 7 November 2026 | Region: ap-south-2 (Hyderabad) | Language: English

1. Objective and scope

Build a repeatable single-account delivery platform in AWS Hyderabad while keeping the already-running Jenkins EC2 instance and its working CI pipeline intact. The new platform adds deployment automation, environment-specific permissions, Kubernetes availability controls, security controls, and recoverable backups.

In scope

- One AWS account and one target region: ap-south-2. Do not hardcode account IDs; read the account ID from aws_caller_identity.
- Existing Jenkins EC2 instance is retained. No public or private IP address is required for IAM role assumption.
- Jenkins obtains its base AWS identity from its EC2 instance profile. The base role assumes separate Dev, QA and Production deployment roles using STS temporary credentials.
- The existing CI job remains unchanged. A separate CD job is cloned or created after exporting and reviewing the existing configuration.
- Where the application is deployed to EKS, worker nodes and application Pods are distributed across Availability Zones using topology spread constraints, replicas, PDBs, probes and autoscaling.
- Terraform provisions IAM and platform resources. Jenkins Configuration as Code (JCasC) manages supported controller settings. Job DSL or a separately cloned job manages job definitions.

Not assumed

- No assumption that an EKS cluster, VPC, artifact buckets, ECR repositories, RDS database, ALB, or KMS keys already exist.
- No assumption that the existing Jenkins EC2 instance has no IAM role. Inspect the attached instance profile before changing anything.
- No assumption that the application is stateless. Stateful storage, database backups and EBS volume placement need application-specific design.
- No claim that one AWS account provides the same security boundary as separate AWS accounts. Dev/QA/Prod separation in one account is logical and permission-based.

Decision to confirm before implementation: whether the application target is EKS. The topology-spread examples in this runbook apply to Pods on Kubernetes/EKS; they do not increase availability for applications deployed only to EC2 or CloudFormation-managed non-Kubernetes services.

2. Target architecture

The target design separates the Jenkins control identity from the deployment identities. Jenkins does not hold permanent AWS access keys. It first receives short-lived instance-profile credentials, then assumes exactly one environment deployment role for a deployment.

Layer	Components	Purpose
Source and CI	Git repository; existing Jenkins CI job; tests; quality and image scans	Build once, test, package and publish an immutable artifact. Keep current CI behaviour unchanged.
Artifact storage	Amazon ECR for container images; S3 for deployment packages and manifests where needed	Store immutable version/tag or digest and promote the same artifact through environments.
Jenkins identity	Existing Jenkins EC2 + attached instance profile + Jenkins base IAM role	Provides the base identity. The role needs only sts:AssumeRole on the three approved target roles.
Deployment identity	payment-service-dev-deployer; payment-service-qa-deployer; payment-service-prod-deployer	Grants environment-specific access to the required bucket, EKS cluster/namespace and deployment operations.

Layer	Components	Purpose
Runtime	EKS cluster or clusters in ap-south-2; node groups across private subnets/AZs; Services/Ingress/ALB	Runs the application with multi-AZ placement, health checks and capacity headroom.
Operations	CloudWatch; CloudTrail; GuardDuty; AWS Backup; EBS snapshots; Velero or CSI snapshots as appropriate	Audit, detect, alert, protect data and test restore procedures.

Identity and request flow

Logical flow

```
Developer pushes code
    -> Existing Jenkins CI job (unchanged)
    -> unit/integration tests + quality/security scans
    -> publish immutable image to ECR (or package to S3)
    -> separate Jenkins CD job
    -> EC2 instance profile credentials (Jenkins base role)
    -> STS AssumeRole to Dev OR QA OR Production deploy role
    -> EKS access entry + Kubernetes RBAC (if EKS target)
    -> Helm/kubectl deploy to the matching environment
    -> health checks, rollout status and deployment record
```

An IAM role assumption alone does not grant Kubernetes API access. For EKS, the assumed deployment role must also be admitted by an EKS access entry (or an existing supported authentication configuration) and then authorized by an EKS access policy or Kubernetes RBAC. Use namespace-scoped permissions where possible. AWS EKS access entries.

3. Inspect the existing Jenkins EC2 instance first

Do not start by replacing its IAM profile. An EC2 instance can have only one attached IAM role at a time. If a role is already attached, replacing it can remove permissions used by the currently working CI pipeline. AWS documents this as a role replacement operation. Attach or replace an IAM role.

3.1 Discover region and instance ID from inside the instance

IMDSv2 commands (run on the Jenkins EC2 host)

```
TOKEN=$(curl -sS -X PUT \
  http://169.254.169.254/latest/api/token \
  -H 'X-aws-ec2-metadata-token-ttl-seconds: 21600')

REGION=$(curl -sS -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/placement/region)

INSTANCE_ID=$(curl -sS -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id)

echo "Region=$REGION Instance=$INSTANCE_ID"
aws sts get-caller-identity
```

3.2 Discover the current attached instance profile

From an AWS CLI identity that can describe EC2

```
aws ec2 describe-instances \
  --instance-ids <JENKINS_INSTANCE_ID> \
  --region ap-south-2 \
  --query 'Reservations[0].Instances[0].IamInstanceProfile.Arn' \
  --output text
```

If the command returns an instance-profile ARN, inspect its profile and role. Also inventory existing role policies before deciding whether to modify, import or replace it. If it returns None, the instance may not have a role attached; create and attach the base role after testing.

3.3 Jenkins inventory to capture before changes

- Jenkins version, Java version, OS, service unit, JENKINS_HOME path and current controller/agent topology.
- All installed plugin IDs and versions; tool installations; global configuration; security realm; authorization strategy; inbound/outbound agent settings; shared libraries.

- Job names and job types: Pipeline, Multibranch Pipeline, Folder, Organization Folder, Freestyle, or Job DSL-generated jobs.
- For the working CI job: SCM URL, branch/ref, credentials ID names (not secret values), script path, triggers/webhooks, parameters, shared libraries, post-actions and artifact handling.
- Existing attached IAM role and all required permissions. Save a copy of the current role policy documents and instance-profile association.

Never paste public IPs, private IPs, access keys, passwords, private keys, Jenkins credential values or unredacted secrets into a prompt or source repository. The IP address is not required for STS role assumption.

4. Safe IAM design for one AWS account

4.1 Resource naming

Resource	Recommended name
Jenkins base role	Jenkins-Base-Role (or retain the existing EC2 role name if one is already in use)
Instance profile	Jenkins-Instance-Profile (only if creating a new profile)
Dev target role	payment-service-dev-deployer
QA target role	payment-service-qa-deployer
Production target role	payment-service-prod-deployer
Shared boundary policy	Deployment-Role-Permission-Boundary (or use three environment-specific boundaries for tighter scoping)
Guardrail policy	Jenkins-Prevent-Long-Lived-Credentials

4.2 Terraform baseline files

Suggested layout

```
terraform/
  versions.tf
  provider.tf
  variables.tf
  data.tf
  iam/
    jenkins-base-role.tf
    jenkins-base-policy.tf
    jenkins-instance-profile.tf
    deployment-boundary.tf
    dev-deployer-role.tf
    dev-deployer-policy.tf
    qa-deployer-role.tf
    qa-deployer-policy.tf
    prod-deployer-role.tf
    prod-deployer-policy.tf
    iam-guardrail.tf
  eks/
    cluster.tf                # only if Terraform will create EKS
    node-groups.tf            # multi-AZ worker node groups
    access-entries.tf         # IAM deployment roles -> EKS permissions
  outputs.tf
  terraform.tfvars.example
```

4.3 Provider and account lookup

provider.tf and data.tf

```
provider "aws" {
  region = var.aws_region
}

data "aws_caller_identity" "current" {}

# variables.tf
variable "aws_region" {
  type    = string
  default = "ap-south-2"
}
```

4.4 Base role trust policy for existing EC2-hosted Jenkins

jenkins-base-role.tf - EC2 trust relationship

```
resource "aws_iam_role" "jenkins_base" {
  name = "Jenkins-Base-Role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
  })
}
```

If your active EC2 instance already uses another role, do not create a second role and assume that both can be attached. Either add the required sts:AssumeRole permission to the existing role and trust that role from the target roles, or deliberately migrate to Jenkins-Base-Role after copying and reviewing all existing permissions. If Terraform is taking over an existing IAM role, import it into state rather than trying to create a duplicate.

4.5 Base role permission to assume only the three target roles

jenkins-base-policy.tf

```
resource "aws_iam_role_policy" "jenkins_assume_deployers" {
  name = "Jenkins-Assume-Target-Roles"
  role = aws_iam_role.jenkins_base.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["sts:AssumeRole"]
      Resource = [
        aws_iam_role.dev_deployer.arn,
        aws_iam_role.qa_deployer.arn,
        aws_iam_role.prod_deployer.arn
      ]
    }]
  })
}
```

4.6 Target role trust policy

Example: qa-deployer-role.tf

```
resource "aws_iam_role" "qa_deployer" {
  name = "payment-service-qa-deployer"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { AWS = aws_iam_role.jenkins_base.arn }
      Action = "sts:AssumeRole"
    }]
  })

  permissions_boundary = aws_iam_policy.qa_boundary.arn
  max_session_duration = 3600
}
```

For this same-account Jenkins-to-role flow, ExternalId is not required for the direct first-party role assumption described here. Use ExternalId when a third-party service or delegated external party assumes a role and the confused-deputy mitigation applies. Do not keep an unused ExternalId credential in Jenkins.

4.7 Permission boundaries are ceilings, not grants

Set permissions_boundary on each target IAM role. Do not attach the boundary as if it were an ordinary permissions policy and do not create an empty duplicate role merely to hold the boundary. The role still needs an identity policy granting deployment actions; the boundary intersects with those permissions and limits their maximum effective set.

Correct Terraform pattern

```
resource "aws_iam_role" "qa_deployer" {
  # ...trust policy...
```

```
  permissions_boundary = aws_iam_policy.qa_boundary.arn
}

# Do not do this as a substitute for permissions_boundary:
# aws_iam_role_policy_attachment.boundary -> role
```

For the strongest environment isolation, create separate Dev, QA and Production boundaries and scope each boundary to its own artifact bucket, stack ARN, EKS cluster and approved PassRole target. A single broad boundary with Resource="*" can leave too much room if someone later adds a broad role policy.

4.8 Deployment permissions to model

- S3 artifact access: only s3:GetObject and s3:GetObjectVersion on the matching environment path. Add KMS decrypt only for the specific key if artifacts use SSE-KMS.
- ECR pull/push: separate CI push permissions from CD pull/deploy permissions. Prefer CI pushing and CD deploying an immutable image digest.
- EKS: eks:DescribeCluster on the intended cluster, plus an EKS access entry and namespace-scoped Kubernetes authorization for the assumed deployment role.
- CloudFormation: include only the actions and stack resources needed if CloudFormation remains part of the deployment design. CloudFormation resource providers can require additional service-specific permissions.
- iam:PassRole: only when the deployment role must pass a specific CloudFormation/EKS service role. Scope the resource to that exact role and use iam:PassedToService conditions where applicable.
- Do not grant iam:*, AdministratorAccess, broad s3:* or unrestricted eks:* to the Jenkins base role.

5. Region, networking and infrastructure foundations

Set the AWS provider and every regional CLI command to ap-south-2. IAM itself is a global service; the region setting applies to regional resources such as VPC, EC2, EKS, ECR, CloudWatch and regional S3 bucket placement.

5.1 Discover enabled Availability Zones rather than hardcoding names

List AZs available to this account

```
aws ec2 describe-availability-zones \
  --region ap-south-2 \
  --filters Name=state,Values=available \
  --query 'AvailabilityZones[].{Zone:ZoneName,ZoneId:ZoneId,State:State}' \
  --output table
```

Select at least two available AZs for an HA application design; use three when available, supported by the service and justified by capacity/cost. Do not assume AZ names are identical across accounts. Use AZ IDs if a cross-account design ever requires consistent physical-zone mapping.

5.2 Recommended network shape

- VPC with public subnets for internet-facing load balancers only, and private subnets for EKS worker nodes and application data services.
- NAT gateways or VPC endpoints sized for required egress. Consider S3, ECR API/DKR, STS, CloudWatch Logs and Secrets Manager endpoints where appropriate.
- Restrict Jenkins inbound access to VPN/bastion/SSM or a tightly scoped corporate source range. Prefer Systems Manager Session Manager over public SSH exposure.
- Use security groups as stateful workload boundaries; use Kubernetes NetworkPolicies for Pod-to-Pod segmentation where supported by the selected CNI/network policy engine.
- Enable IMDSv2 on the Jenkins EC2 instance and avoid granting instance metadata access to untrusted containers or processes.
- Encrypt EBS volumes, ECR images where applicable, S3 buckets and RDS storage with KMS-managed encryption. Use TLS for service endpoints.

5.3 Region guardrails

terraform.tfvars.example

```
aws_region = "ap-south-2"
```

```
# Replace examples with bucket names that actually exist in ap-south-2.
dev_artifact_bucket  = "<dev-artifact-bucket>"
qa_artifact_bucket   = "<qa-artifact-bucket>"
prod_artifact_bucket = "<prod-artifact-bucket>"
```

Before apply, verify the region is enabled for this account, all selected Availability Zones are available, and the required instance types/service quotas exist in ap-south-2. The runbook does not assume specific AZ names or capacity.

6. EKS high availability and topology spread

Topology spread constraints influence how the Kubernetes scheduler places Pods across failure domains such as zones and nodes. They are a scheduling rule, not a substitute for replicas, healthy nodes, autoscaling, sufficient capacity, probes, disruption budgets or a resilient data layer. Kubernetes topology spread documentation.

6.1 Multi-AZ worker design

- Create private subnets in at least two available AZs and place EKS managed node groups across those subnets. AWS managed node groups span the subnets provided when the node group is created.
- Keep enough desired capacity in each AZ to tolerate a node or AZ loss. A topology rule cannot schedule into an AZ with no eligible nodes or no spare capacity.
- Separate system workloads from application workloads using dedicated node groups, labels/taints and tolerations where justified.
- Use Cluster Autoscaler or Karpenter with disruption and consolidation settings appropriate to availability objectives.
- For stateful applications backed by EBS volumes, plan for AZ-bound volume attachment and storage topology. AWS recommends multiple AZ-scoped node groups in certain stateful EBS + Cluster Autoscaler scenarios.

6.2 Deployment manifest example

k8s/payment-service/deployment.yaml - key fields

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  namespace: payment-service-qa
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: payment-service
  template:
    metadata:
      labels:
        app: payment-service
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector:
            matchLabels:
              app: payment-service
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels:
              app: payment-service
      containers:
        - name: payment-service
          image: <account>.dkr.ecr.ap-south-2.amazonaws.com/payment-service:<immutable-tag>
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet: { path: /health/ready, port: 8080 }
          livenessProbe:
```

```
          httpGet: { path: /health/live, port: 8080 }
        resources:
          requests: { cpu: 250m, memory: 512Mi }
          limits: { cpu: "1", memory: 1Gi }
```

This example assumes the application exposes the shown health endpoints and the labelSelector matches the Pods in the Deployment. Adjust the endpoints, port, image URI, requests, limits and namespace to the real application. Do not paste this manifest into production without validating its requirements.

6.3 Pod disruption budget

k8s/payment-service/pdb.yaml

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: payment-service
  namespace: payment-service-qa
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: payment-service
```

With three replicas and minAvailable: 2, voluntary disruptions should preserve at least two healthy Pods. A PDB does not protect against every involuntary failure, and it can block node drains if there is insufficient healthy capacity. Validate rollout, upgrade and drain procedures.

6.4 HPA example

k8s/payment-service/hpa.yaml

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payment-service
  namespace: payment-service-qa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 65
```

HPA requires a working metrics pipeline, such as metrics-server for CPU/memory resource metrics. Set realistic minReplicas, maxReplicas, CPU requests and capacity limits from load testing. A high-availability design should have enough spare node capacity for scale-up and rescheduling after a failure.

6.5 Other availability controls

- Kubernetes Service with readiness-aware endpoints; ALB/NLB health checks; at least two AZs for load balancer subnets when required by the controller/service.
- Startup probes for slow-starting applications; readiness probes for traffic eligibility; liveness probes for deadlock/recovery detection.
- Rolling updates with maxUnavailable 0 or a validated safe value; maxSurge at least 1 if capacity allows.
- Anti-affinity can complement topology spread, but avoid hard anti-affinity rules that make Pods unschedulable during capacity pressure.
- For stateful workloads, use supported storage replication/backup patterns. A single-AZ EBS volume is not made multi-AZ merely by spreading its Pod.

7. Security baseline

7.1 AWS identity and access

- Use EC2 instance profile credentials for Jenkins; do not store AWS access keys in Jenkins credentials or source code.
- Use a base role that can only sts:AssumeRole to the three deployment roles. Target roles trust only the Jenkins base role ARN.
- Use one target role per environment. Scope S3 buckets/prefixes, CloudFormation stack ARNs, ECR repositories, EKS clusters and PassRole targets to that environment.
- Use IAM permissions boundaries as ceilings. Attach boundary ARNs on the IAM role resource itself. Test effective permissions with policy simulation and actual non-production deployments.
- Add explicit-deny guardrails for prohibited operations such as creating IAM users/access keys if required by your security policy. Validate that the guardrail does not break approved automation.
- For EKS access, use EKS access entries and namespace-scoped authorization where possible rather than giving deployment roles cluster-admin.

7.2 Workload and supply-chain security

- Keep the working CI tests; add or verify SAST/dependency scan, secret scan, container image scan (for example Trivy), SBOM generation and artifact provenance where appropriate.
- Use immutable image digests or immutable version tags. Promote the same artifact from Dev to QA to Production; do not rebuild different images for each environment.
- Run containers as non-root, drop Linux capabilities, use read-only root filesystems where the app supports it, set seccomp and resource requests/limits.
- Store application secrets in AWS Secrets Manager or Parameter Store with KMS encryption; mount/inject at runtime through an approved mechanism. Do not put plaintext secrets in JCasC, Helm values, Git, build logs or PDF documents.
- Restrict production approvals to an authorized group; protect deployment branches and require review for IAM, Jenkinsfile, Terraform and Kubernetes manifest changes.

7.3 Logging, audit and detection

- Enable CloudTrail management-event logging and protect the trail destination; alert on IAM policy/role changes and root-account events.
- Use CloudWatch Logs/metrics and alarms for Jenkins service health, EC2 disk/CPU/memory, EKS control-plane logs, Pod restarts, failed deployments and load balancer target health.
- Enable GuardDuty and consider AWS Config rules/security posture monitoring. Send critical alerts to a monitored channel/on-call path.
- Use Kubernetes audit logs, RBAC review and NetworkPolicies where supported. Avoid logging tokens, passwords, connection strings or customer data.

8. Preserve and clone the existing Jenkins CI configuration

The existing CI pipeline is already working. Treat it as a production dependency: export and back it up first, then create a separate CD job. Do not overwrite or reconfigure the working CI job as part of the first rollout.

8.1 What JCasC can and cannot do

Jenkins Configuration as Code stores supported controller configuration in YAML. The official Jenkins documentation describes the Configuration as Code page as a way to view/export the current controller configuration. JCasC does not, by itself, guarantee a complete clone of every job definition, plugin binary, build record, workspace, external secret or artifact. See Jenkins JCasC documentation.

8.2 Non-destructive extraction sequence

- Install the Configuration as Code plugin only after checking Jenkins and plugin compatibility; take a backup first.
- Open Manage Jenkins -> Configuration as Code -> View Configuration; download the YAML export. Review it and remove/sanitize secret values before committing to private source control.
- Export the installed plugin list with versions. Reproduce plugins through a pinned plugin catalog or plugins.txt in a test controller; JCasC configures supported settings but is not a plugin installer.
- Back up job configuration. For each job, capture its config.xml and record SCM URL, branch, credentials IDs, Jenkinsfile path, parameters, triggers and shared-library configuration. Never export the secret values.
- Create a new job named payment-service-cd by using Copy from in Jenkins or Job DSL. Keep payment-service-ci unchanged. Point the new job to a separate CD pipeline file or a separate repository path.
- If the source job is Multibranch Pipeline, create a parallel Multibranch Pipeline job/folder and configure its branch source and script path separately. Do not assume a normal Pipeline copy is equivalent.
- Use the existing credential IDs only where they are appropriate. Prefer the EC2 instance profile for AWS. Recreate non-AWS credentials through the approved secret store/credentials process.

JCasC is best for controller settings; use Job DSL or a separately cloned job definition for jobs. Do not blindly restore an old JENKINS_HOME onto the live controller or copy all credentials/secrets into Git.

8.3 Minimal JCasC starter file

jenkins/casc/jenkins.yaml - safe starter, extend from the sanitized export

```
jenkins:
  systemMessage: "Managed as code. Changes are reviewed in source control."
  # Set controller executors to zero only after build agents are configured
  # and the existing CI pipeline has been tested on those agents.
  numExecutors: 1
  scmCheckoutRetryCount: 2
```

This is a deliberately small starter, not a full export of your current Jenkins. Add only settings that match installed plugin versions and your existing authentication/authorization design. Do not copy a generated credentials section until every secret reference has been reviewed. Use CASC_JENKINS_CONFIG to point Jenkins at the controlled YAML directory. Validate on a test controller before loading it on the current controller.

8.4 Job configuration options

Current job type	Recommended clone method
Pipeline from SCM	Copy the job, retain SCM details/credential IDs, and change the script path to Jenkinsfile.cd or use a separate CD repo.
Pipeline script in UI	Export the job config and script securely; move scripts to SCM, then create a separate CD job.
Multibranch Pipeline	Create a second Multibranch job with equivalent branch-source configuration and a CD-specific Jenkinsfile path.
Freestyle / folder / Job DSL	Capture job config and plugins; reproduce with Job DSL or carefully copy the item. Verify triggers, permissions and build steps.

9. CD pipeline design

Keep CI and CD as separate jobs in the initial rollout. CI builds/tests/scans and publishes an immutable artifact. CD selects a target environment, obtains temporary credentials by assuming one target role, applies the same artifact, and verifies health. Production requires explicit approval.

9.1 Pipeline stages

Stage	Required behaviour
Pre-flight	Validate branch, artifact digest, deployment parameters, AWS region, role ARN and cluster name.
Environment selection	Map dev/qa/prod to an exact target role, namespace, EKS cluster and values file. Never accept arbitrary role ARNs from a user parameter.
Production approval	Manual approval by an authorized group; log approver and build/artifact reference.
Assume role	Use AWS CLI default EC2 credentials to call sts:AssumeRole; export temporary credentials only inside one controlled shell step; disable shell tracing.
EKS authentication	Update kubeconfig for the target cluster. Verify current context and `kubectl auth can-i` before deploy.
Deploy	Use Helm upgrade/install or controlled kubectl apply with immutable image digest and environment-specific values.
Verify	Check rollout status, Pod readiness, PDB status, service endpoints and application smoke test. Fail on unhealthy rollout.
Record/rollback	Record artifact digest, role session, cluster, namespace and release revision. Roll back to the previous known-good release on a failed rollout according to policy.

9.2 Jenkinsfile CD shell pattern

Jenkinsfile.cd - core same-account STS pattern (adapt cluster and release names)

```
pipeline {
  agent any
  parameters {
    choice(name: 'DEPLOY_ENV', choices: ['dev', 'qa', 'prod'])
  }
  environment {
    AWS_REGION = 'ap-south-2'
    APP_NAME = 'payment-service'
    EKS_CLUSTER_NAME = 'payment-platform'
  }
  stages {
    stage('Select target') {
      steps {
        script {
          def roles = [
            dev: 'payment-service-dev-deployer',
            qa: 'payment-service-qa-deployer',
            prod: 'payment-service-prod-deployer'
          ]
          env.TARGET_ROLE = roles[params.DEPLOY_ENV]
          env.NAMESPACE = "payment-service-${params.DEPLOY_ENV}"
        }
      }
    }
    stage('Production approval') {
      when { expression { params.DEPLOY_ENV == 'prod' } }
      steps { input message: 'Approve production deployment?' }
    }
    stage('Assume role and deploy') {
      steps {
        sh '''#!/usr/bin/env bash
          set -euo pipefail
          set +x
          ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
          ROLE_ARN="arn:aws:iam::${ACCOUNT_ID}:role/${TARGET_ROLE}"
          CREDS=$(aws sts assume-role \
            --role-arn "$ROLE_ARN" \
            --role-session-name "jenkins-${BUILD_NUMBER}" \
            --duration-seconds 3600 \
            --query 'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]' \
            --output text)
          read -r AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN <<< "$CREDS"
          export AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN
          aws sts get-caller-identity
          aws eks update-kubeconfig --name "$EKS_CLUSTER_NAME" --region "$AWS_REGION"
          kubectl auth can-i patch deployments -n "$NAMESPACE"
          helm upgrade --install "$APP_NAME" ./charts/payment-service \
            --namespace "$NAMESPACE" --create-namespace \
            --set image.digest="$IMAGE_DIGEST" \
            -f "values-${DEPLOY_ENV}.yaml"
```

```
          kubectl rollout status deployment/payment-service \
            -n "$NAMESPACE" --timeout=5m
        '''
      }
    }
  }
}
```

The Jenkinsfile is a pattern, not a drop-in script: define IMAGE_DIGEST from the successful CI artifact, set the real cluster/Helm chart/values paths, and implement the EKS access entry/RBAC before running it. `kubectl auth can-i` must check the actual verbs/resources required by the Helm release, not just deployments. The same role cannot perform EKS API operations unless its IAM policy and EKS access configuration allow them.

10. Backup, restore and disaster recovery

Backups are only useful when restore has been tested. Define RPO (maximum acceptable data loss) and RTO (maximum recovery time) with the application and business owners before selecting retention and schedule.

10.1 Jenkins controller backup

The Jenkins documentation explains that backing up the whole JENKINS_HOME can preserve the controller, while selective backups require more deliberate restore planning. Job configuration resides under jobs; plugin packages are under plugins; global configuration files are in JENKINS_HOME. Jenkins backup and restore documentation.

- Back up the Jenkins EBS volume using a controlled, application-consistent process. For strict consistency, quiesce/stop Jenkins or use a suitable filesystem/VSS-like snapshot approach before the snapshot; coordinate any downtime.
- Also back up JENKINS_HOME, JCasC YAML, plugin inventory/versions, Job DSL, job definitions, shared libraries metadata, tool configuration and systemd/environment configuration.
- Protect the Jenkins secrets directory and master key material. They are required to decrypt stored credentials during restore, and must be encrypted and access-restricted separately. Never store these in an ordinary Git repository.
- Do not normally back up workspaces as a source of truth; they can generally be recreated by SCM checkout. Decide separately whether archived artifacts/build logs must be retained.
- Store encrypted backups in a dedicated S3 bucket with versioning, least-privilege access, lifecycle retention and optionally Object Lock where compliance requires immutability.
- Consider cross-region copies for disaster recovery, for example to ap-south-1, if business RPO/RTO requires a second-region copy. This is optional and is not part of the Hyderabad-only runtime footprint.

10.2 Application and platform backups

Asset	Backup approach	Restore test
Terraform state	Remote encrypted S3 backend with versioning; state locking supported by the selected backend design; restrict access tightly.	Restore a prior version in a controlled workspace; verify no concurrent apply.
EKS objects	Git as source of truth; optional Velero backup for cluster-scoped/namespaced objects and selected persistent data.	Restore into a non-production cluster/namespace and validate dependencies.
EBS-backed PVCs	EBS CSI snapshots or AWS Backup where supported; coordinate with application consistency needs.	Restore a PVC/volume and run application-level validation.
RDS / database	Automated backups and point-in-time recovery; snapshots and cross-region copies if required.	Perform a restore to an isolated DB and run integrity/application checks.
ECR images / artifacts	Immutable image tags/digests, lifecycle policy and protected artifact storage.	Deploy a retained digest into a test environment.
Jenkins	EBS snapshots plus protected JENKINS_HOME/JCasC/job/plugin backup.	Restore a test controller and run the existing CI job plus a CD dry run.

10.3 Illustrative retention (confirm business policy)

- Daily backups: 14 days; weekly backups: 8 weeks; monthly backups: 12 months. These are starting examples only, not a compliance recommendation.
- Keep at least one protected copy outside the primary failure domain if required by the agreed DR plan.
- Run a restore exercise monthly for critical Jenkins configuration and quarterly for a broader platform/application recovery. Record actual RPO/RTO achieved and remediation actions.

10.4 Jenkins HA limitation

A single Jenkins controller EC2 instance remains a single point of failure. JCasC and backups make it reproducible/recoverable, but they do not create active-active Jenkins. Do not let two controllers write to the same JENKINS_HOME. A practical first step is a tested warm-standby/restore procedure in another AZ, a documented DNS/ALB cutover, and scalable build agents. Keep the Jenkins controller on one active writer at a time.

11. Step-by-step implementation order

Step 1 - Record the current state - Capture EC2 instance ID, region, current instance profile/role, Jenkins version, JENKINS_HOME, plugins, job types and current CI job details. Save screenshots/exports and redact secrets.

Step 2 - Back up Jenkins - Take an application-consistent backup of JENKINS_HOME and an EBS snapshot. Securely preserve encryption/master-key material. Verify that the backup can be read and document the restore path.

Step 3 - Confirm Hyderabad foundations - Verify ap-south-2 is enabled, discover available AZs, identify VPC/subnets, confirm service quotas and check that required S3/ECR/EKS resources are or will be in the intended region.

Step 4 - Decide the active Jenkins role - If an IAM role is already attached, preserve it and add only the required sts:AssumeRole permission or plan a controlled role migration. Do not blindly replace the instance profile.

Step 5 - Build Terraform foundation - Configure remote state, encryption, version pinning, variables, provider and data.aws_caller_identity. Run terraform fmt, validate and plan. Review all IAM diffs.

Step 6 - Create target IAM roles - Create Dev, QA and Production trust policies, scoped permissions and permission boundaries. Confirm that target trust names the actual role attached to the Jenkins EC2 instance.

Step 7 - Attach/verify Jenkins identity - If a new profile is required, attach it through EC2 Modify IAM role only after preserving existing permissions. Verify aws sts get-caller-identity and test sts:AssumeRole against Dev only.

Step 8 - Export Jenkins configuration - Install/use JCasC after compatibility review; export sanitized controller YAML; capture plugin versions and job XML/metadata. Store only reviewed non-secret configuration in private source control.

Step 9 - Clone the pipeline safely - Copy the existing CI job to a new CD job or create a separate Multibranch job. Keep the CI job unchanged. Confirm CI still builds and publishes the same artifact.

Step 10 - Build/prepare EKS networking - Provision or verify the EKS control plane, private subnets and node groups across multiple available AZs. Set EKS access authentication and add least-privilege access entries for deployment roles.

Step 11 - Prepare Kubernetes workload - Create namespaces, RBAC, quotas/limits, NetworkPolicies where supported, Deployment, Service, Ingress, topology spread constraints, PDB, HPA and probes. Test scheduling with enough nodes in each AZ.

Step 12 - Deploy to Dev - Use a fixed artifact digest, assume the Dev role, deploy with Helm, verify rollout and smoke tests, inspect Pod distribution by zone and node.

Step 13 - Promote to QA - Promote the exact same artifact digest; run integration/performance/security checks and verify recovery from a Pod/node disruption.

Step 14 - Production release - Require approval, validate change window and rollback target, deploy the already-tested digest, check rollout/PDB/alerts and record release metadata.

Step 15 - Turn on backup/monitoring controls - Enable CloudTrail, CloudWatch alarms/log retention, AWS Backup schedules or equivalent snapshots, S3 versioning/lifecycle and restore tests. Define owners and on-call process.

Step 16 - Disaster recovery exercise - Restore Jenkins to an isolated test host, recover a sample app database/PVC if applicable, redeploy from Git/Terraform and measure RTO/RPO. Update runbooks from observed results.

12. Validation checklist and commands

Terraform checks

Run from the Terraform root directory

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform show tfplan
# Apply only after reviewing the plan and obtaining the required approval.
terraform apply tfplan
```

Jenkins and STS checks

Run from the Jenkins agent that executes CD

```
aws sts get-caller-identity
aws sts assume-role \
  --role-arn arn:aws:iam::<ACCOUNT_ID>:role/payment-service-dev-deployer \
  --role-session-name jenkins-dev-validation \
  --duration-seconds 900
```

EKS checks

After assuming the target deploy role

```
aws eks update-kubeconfig --name <CLUSTER_NAME> --region ap-south-2
kubectl config current-context
kubectl auth can-i get deployments -n payment-service-dev
kubectl get nodes -L topology.kubernetes.io/zone
kubectl get pods -n payment-service-dev -o wide
kubectl get pdb -n payment-service-dev
kubectl rollout status deployment/payment-service -n payment-service-dev
```

Expected validation outcomes

- Base identity is the intended existing Jenkins EC2 role; no IAM user access keys appear in the pipeline.
- Dev role cannot read QA/Production artifact paths or deploy to their namespaces; QA cannot deploy to Production; Production requires approval.
- EKS access entry and RBAC authorize only the required namespace-level resources and verbs.
- Three or more app replicas are distributed across available zones/nodes as expected when capacity exists; a single-zone/node failure does not take every replica offline.
- PDB does not block normal maintenance; HPA can scale; node group autoscaling has enough capacity and quotas.
- The existing CI job still runs successfully and its behaviour is unchanged; the new CD job deploys a known immutable artifact.
- A test restore of Jenkins configuration and at least one application data path succeeds.

13. Common failure modes

Symptom	Likely cause / check
AccessDenied on sts:AssumeRole	Base role lacks sts:AssumeRole on exact target role ARN, or target trust policy does not trust the actual attached EC2 role.
Jenkins loses AWS access after profile change	The previous role had permissions required by CI. Restore prior profile or merge required permissions before migration.
kubectl Unauthorized / Forbidden	IAM role assumption succeeded but EKS access entry/authentication or Kubernetes RBAC is missing or too broad/narrow.
Pods Pending due to topology spread	Insufficient nodes/capacity in one or more eligible zones, selector mismatch, taints/affinity, strict DoNotSchedule or quota shortage.
PDB blocks node drain	Not enough healthy replicas. Increase capacity, inspect readiness, or adjust replica/PDB policy based on availability requirement.
Helm rollout times out	Image pull error, wrong digest, missing secret, failing readiness probe, insufficient capacity, bad values or missing ConfigMap/ServiceAccount.
JCasC reload fails	Plugin missing/incompatible, malformed YAML, unsupported configurator key, duplicate keys or secret interpolation issue. Test on a non-production controller.
Jenkins restore cannot decrypt credentials	Missing or mismatched Jenkins secrets/master key material. Restore the protected key material from the same backup set.
CloudFormation fails after role assumption	Target role does not include the service-specific permissions required by the template or the exact PassRole/KMS permissions.

14. Decisions and values to confirm before production

Decision	What must be confirmed
AWS account	Account ID is discovered dynamically; verify all resources are in the intended one account.
Existing EC2 IAM role	Current instance profile name and role ARN; choose preserve/extend or controlled replacement.
Network	Existing VPC/subnet IDs, private/public subnet strategy, NAT/VPC endpoints, security-group rules.
EKS target	Existing cluster name or new cluster; Kubernetes version supported by EKS at implementation time; authentication mode; private/public API endpoint policy.
Environment model	Separate EKS clusters or one cluster with Dev/QA/Prod namespaces. Separate clusters provide stronger isolation but cost more. Namespaces are not AWS account boundaries.
Artifact flow	ECR repository, immutable image digest, S3 artifacts if needed, retention and promotion rules.
Jenkins clone	Existing job type, source of Jenkinsfile, shared libraries, plugin set, credentials IDs, webhooks and job permissions.
Availability	Number of AZs, replicas, node group capacity, HPA limits, PDB, service health checks and disruption test.
Backup/DR	RPO, RTO, backup frequency/retention, cross-region copy, restore owner and test cadence.
Application data	RDS or other datastore, encryption key, backup/restore process, persistent volumes and consistency requirements.

Do not fill unknown values with guesses. The implementation should use variables and clearly marked placeholders, and the first Terraform plan should be reviewed against the actual AWS account before applying.

15. Authoritative references

Jenkins Configuration as Code
https://www.jenkins.io/doc/book/managing/casc/
JCasC export, YAML configuration and controller configuration management.

Jenkins backup and restore
https://www.jenkins.io/doc/book/system-administration/backing-up/
JENKINS_HOME, jobs, configuration and backup considerations.

Jenkins Configuration as Code plugin
https://plugins.jenkins.io/configuration-as-code/
Plugin/security considerations and plugin installation distinction.

AWS EC2: attach an IAM role to an instance
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/attach-iam-role.html
Instance profile attachment and role replacement behaviour.

Kubernetes Pod topology spread constraints
https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/
Topology spread fields and failure-domain scheduling.

Amazon EKS managed node groups
https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html
Managed node groups and subnet/AZ behaviour.

Amazon EKS access entries
https://docs.aws.amazon.com/eks/latest/userguide/access-entries.html
IAM principal to Kubernetes access mapping.

Amazon EKS subnet best practices
https://docs.aws.amazon.com/eks/latest/best-practices/subnets.html
Multi-AZ subnet design and load balancer subnet considerations.

This runbook was prepared from the requirements in the conversation and current official product documentation. Validate service availability, version compatibility, pricing, quotas and security controls in the target AWS account before implementation. AWS and Kubernetes documentation can change.